

SNMP / LDAP / RPC Enumeration — Study Guide

These three protocols round out the "classic enumeration trio" alongside SMB (which you've already covered via NSE, Enum4linux, and SMBMap). Each exposes a different slice of an organization's internal structure — SNMP leaks device/network config, LDAP leaks the entire Active Directory structure, and RPC is the underlying transport that many Windows enumeration tools (including Enum4linux) actually ride on top of.

PART 1: SNMP Enumeration

1. What SNMP Is

SNMP (Simple Network Management Protocol), UDP port 161 (traps on 162), is used to monitor and manage network devices — routers, switches, printers, servers. It's designed for administrators to remotely query device status, but weak authentication (especially default/guessable **community strings**) makes it a frequent misconfiguration finding.

2. SNMP Versions and Why They Matter

Version	Authentication	Notes
SNMPv1	Plaintext community string	Oldest, weakest, still surprisingly common on legacy gear
SNMPv2c	Plaintext community string	Most widely deployed version in practice
SNMPv3	Actual username/password + optional encryption	The only version with real security — if you see this, brute-forcing community strings is pointless

Default community strings worth knowing (the classic starting point):

```
public    → typically read-only access
private  → typically read-write access
```

Finding SNMP still configured with `public` / `private` defaults is one of the most common "quick win" findings in network-focused engagements.

3. Basic Enumeration

```
# Check if SNMP is running and community string works
snmpwalk -v2c -c public target

# Query a specific OID (Object Identifier - SNMP's addressing scheme)
snmpwalk -v2c -c public target 1.3.6.1.2.1.1.1.0 # system description
```

Community string brute-forcing:

```
onesixtyone -c community_strings.txt target
onesixtyone -c community_strings.txt -i targets.txt # multiple targets
```

4. Useful OIDs (What to Actually Query)

OID	Reveals
1.3.6.1.2.1.1.1.0	System description (OS, version)
1.3.6.1.2.1.1.5.0	System hostname
1.3.6.1.2.1.25.1.6.0	System processes
1.3.6.1.2.1.25.4.2.1.2	Running software list
1.3.6.1.4.1.77.1.2.25	Windows user accounts (via SNMP on Windows hosts)
1.3.6.1.2.1.6.13.1.3	TCP listening ports

```
snmpwalk -v2c -c public target 1.3.6.1.2.1.25.1.6.0 # list running processes
snmpwalk -v2c -c public target 1.3.6.1.4.1.77.1.2.25 # list Windows users (older Windows/net-snmp)
```

5. Automated Tools

```
# snmp-check - clean, organized full enumeration in one command
snmp-check target -c public

# nmap NSE
nmap -sU -p161 --script=snmp-brute target # brute
community strings
nmap -sU -p161 --script=snmp-sysdescr target # system description
nmap -sU -p161 --script=snmp-processes target # running processes
nmap -sU -p161 --script=snmp-win32-users target # Windows users (on legacy SNMP-enabled Windows)
```

6. Why SNMP Findings Matter

An accessible **read-write** community string is a severe finding — SNMP write access on network devices can allow reconfiguration (redirecting traffic, disabling logging, even, on some older Windows hosts, remote command execution via specific MIBs). Even **read-only** access is valuable for recon — process lists, installed software, and network topology info all feed directly into your next steps.

PART 2: LDAP Enumeration

1. What LDAP Is

LDAP (Lightweight Directory Access Protocol), TCP port 389 (636 for LDAPS/encrypted), is how Active Directory stores and serves its entire directory structure — users, groups, computers, organizational units (OUs), group policies, trust relationships. If SMB/RPC tell you *what's on a box*, LDAP tells you *the entire org chart of the domain*.

2. Anonymous LDAP Binds

Like SMB null sessions, many AD environments (especially older/misconfigured ones) allow an **anonymous bind** — querying the directory with no credentials at all:

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com"
```

Flag	Meaning
-x	Simple authentication (as opposed to SASL)
-H	LDAP URI
-b	Base DN (Distinguished Name) — the search starting point in the directory tree
-D	Bind DN (for authenticated queries)
-w	Password

3. Authenticated Queries (Far More Common in Modern AD)

```
ldapsearch -x -H ldap://target -D "CN=jsmith,CN=Users,DC=example,DC=com" -w 'Password123' -b "dc=example,dc=com"
```

4. Useful Filters — Pulling Specific Information

```
# All user accounts
```

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(objectClass=user)"
```

```
# All computer accounts (useful for mapping the domain's machines)
```

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(objectClass=computer)"
```

```
# Domain Admins group membership
```

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(cn=Domain Admins)"
```

```
# Accounts with Kerberos pre-auth disabled (ASREPRoastable - a known AD attack path)
```

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(userAccountControl:1.2.840.113556.1.4.803:=4194304)"
```

```
# Service accounts (often tied to Kerberoasting)
```

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(servicePrincipalName=*)"
```

5. Windows/Kali Native Tooling

```
# ldapdomaindump - produces clean HTML/JSON/CSV reports of
the whole domain
ldapdomaindump -u 'DOMAIN\jsmith' -p 'password' target

# windapsearch - Python tool built specifically for AD LDAP
recon
windapsearch.py --dc-ip target -u 'jsmith@example.com' -p
'password' --da # find Domain Admins
```

6. LDAP's Role in Modern AD Attack Chains

LDAP is what tools like **BloodHound** and **PowerView** query under the hood — when BloodHound builds its attack-path graphs, it's fundamentally running structured LDAP queries and correlating the results. Understanding raw `ldapsearch` first makes BloodHound feel far less like a black box once you get there.

```
# nmap NSE for a quick first LDAP check
nmap -p389 --script=ldap-search target
nmap -p389 --script=ldap-rootdse target # reveals domain
naming context, server info
```

7. Why LDAP Findings Matter So Much

A working anonymous or low-privilege LDAP bind can reveal the **entire domain structure** in one query — every username, every group, every computer, OU hierarchy, and password policy. This is frequently the single most information-dense enumeration step in an AD-focused engagement, and it's exactly why locking down anonymous LDAP binds is a standard AD hardening recommendation.

PART 3: RPC Enumeration

1. What RPC Is

RPC (Remote Procedure Call), specifically **MSRPC** on Windows (typically over TCP port 135, with dynamic high ports for the actual service, or tunneled through SMB named pipes on 445), is the underlying mechanism Windows uses for a huge range of inter-process/inter-host communication — including much of what Enum4linux and SMB enumeration tools actually query behind the scenes.

Key concept: when you ran `enum4linux -a` earlier, a large portion of what it was doing (user enumeration, RID cycling, domain info) was literally making **RPC calls** through an SMB null session. RPC and SMB are deeply intertwined on Windows — this section is about interacting with RPC directly and more granularly than Enum4linux's automated sweep.

2. `rpcclient` — The Core Manual Tool

```
rpcclient -U "" target           # null session
rpcclient -U "jsmith%password" target # authenticated
```

Once inside the interactive `rpcclient` shell, useful commands:

```
srvinfo           # server info (OS version, server type)
enumdomusers      # list all domain users
enumdomgroups     # list all domain groups
queryuser <RID>   # detailed info on a specific user
querygroup <RID>  # detailed info on a specific group
querydomaininfo   # domain info (incl. password policy)
netshareenum      # list shares
lsaquery          # query LSA policy info (domain SID)
lookupnames <username> # resolve a username to a SID
lookupsids <SID>    # resolve a SID to a username
enumprivs         # list privileges on the system
getdompwininfo    # password policy details
```

3. Practical Example — Manual RID Cycling With `rpcclient`

This is genuinely worth doing manually at least once, since it demystifies exactly what Enum4linux automates:

```
rpcclient -U "" target
> lookupsids S-1-5-21-1234567890-1234567890-1234567890-500
```

```
> lookupsids S-1-5-21-1234567890-1234567890-1234567890-501
> lookupsids S-1-5-21-1234567890-1234567890-1234567890-1000
```

Each incrementing RID (500, 501, 1000, 1001...) potentially resolves to a different account — 500 is always the built-in Administrator, 501 is always Guest, and anything 1000+ is typically a regular created account. This is the exact mechanism behind Enum4linux's automated user enumeration.

4. **impacket** Suite — RPC/SMB Tooling Beyond rpcclient

The **impacket** collection of Python tools is essential once you move past basic enumeration — several scripts specifically target RPC/SMB:

```
# lookupsid.py - RID cycling automated, similar concept to
manual rpcclient loop above
python3 lookupsid.py DOMAIN/jsmith:password@target

# samrdump.py - dumps SAM database info via RPC
python3 samrdump.py DOMAIN/jsmith:password@target

# psexec.py - remote command execution via SMB/RPC (needs a
dmin creds)
python3 psexec.py DOMAIN/administrator:password@target
```

5. Nmap NSE for RPC

```
nmap -p135 --script=msrpc-enum target
nmap --script=rpcinfo target # for Linux/Unix R
PC services (portmapper, NFS-related)
```

Note the distinction: Linux/Unix RPC (`rpcinfo`, `showmount`, tied to NFS/portmapper on port 111) is a *different protocol family* from Windows MSRPC (port 135) — same acronym, different underlying implementation. Worth not conflating the two.

```
# Linux/Unix RPC enumeration (relevant on Metasploitable-st
yle boxes)
rpcinfo -p target
```

```
showmount -e target          # list NFS exports if rpcbind/NFS is running
```

6. Why RPC Matters as Its Own Category

Understanding RPC directly matters because:

- It's the actual transport layer many "SMB enumeration" tools use — knowing this connects your Enum4linux/SMBMap knowledge to what's really happening on the wire
- `rpcclient` lets you ask far more granular, specific questions than an automated tool's fixed checklist covers
- Several classic Windows RCE vulnerabilities (MS08-067, PrintNightmare's RPC component) are RPC-based, so understanding the protocol helps contextualize why those exploits work

Putting All Three Together — Combined Recon Workflow

```
# 1. Nmap - identify which of these services are even present
nmap -sU -p161 target          # SNMP
nmap -p389,636 target          # LDAP
nmap -p135,445 target          # RPC/SMB

# 2. SNMP - default community string check
onesixtyone -c community_strings.txt target
snmpwalk -v2c -c public target 1.3.6.1.2.1.1.1.0

# 3. LDAP - anonymous bind check, domain structure dump
ldapsearch -x -H ldap://target -b "dc=example,dc=com"
ldapdomaindump -u '' -p '' target

# 4. RPC - manual/automated enumeration
rpcclient -U "" target
enum4linux -a target          # ties SMB + RPC checks together automatically
```

```
# → Correlate findings: usernames from RPC/LDAP feed password spraying,  
#   SNMP process/software info feeds vulnerability research,  
#   LDAP group data reveals privilege escalation targets (Domain Admins, etc.)
```

Kill Chain / ATT&CK Mapping

Protocol	Framework Mapping
SNMP	T1602.001 (Data from Configuration Repository: SNMP), T1046 (Network Service Discovery)
LDAP	T1087.002 (Account Discovery: Domain Account), T1069.002 (Permission Groups Discovery: Domain Groups), T1482 (Domain Trust Discovery)
RPC	T1046 (Network Service Discovery), T1135 (Network Share Discovery) — underlies much of the SMB-related technique coverage from your Enum4linux/SMBMap guides

Defensive Angle (SOC Relevance)

SNMP

- Change default community strings immediately (`public` / `private` should never appear in a production environment)
- Migrate to SNMPv3 wherever device support allows — actual authentication + encryption
- Restrict SNMP access via ACLs to only management-network IPs, never expose it broadly

LDAP

- Disable anonymous LDAP binds (a standard AD hardening baseline, but still frequently found enabled on older environments)
- Enable **LDAP signing/channel binding** to prevent relay-style attacks
- Monitor for high-volume LDAP queries from a single source — a strong signal of BloodHound-style domain enumeration in progress

RPC

- Restrict RPC endpoint mapper exposure where possible (network segmentation)
 - Monitor for RID cycling patterns (sequential `lookupsids` / `queryuser` calls) — same detection logic as the Enum4linux RID cycling note from your earlier SMB guide
 - Keep systems patched — RPC-based RCEs (MS08-067, PrintNightmare) remain some of the most impactful vulnerabilities in Windows history precisely because RPC is so foundational
-

Legal & Ethical Notes

All three protocols here are **active enumeration techniques** — they connect directly to the target, generate authentication logs (even failed/anonymous ones), and can trigger IDS/IPS or SIEM alerts, especially LDAP's high query volume during a full domain dump. Same rule as every active technique covered so far: only run these against systems you own or are explicitly authorized to test.

Quick Reference Cheat Sheet

SNMP

```
snmpwalk -v2c -c public target
onesixtyone -c community_strings.txt target
snmp-check target -c public
nmap -sU -p161 --script=snmp-brute target
```

LDAP

```
ldapsearch -x -H ldap://target -b "dc=example,dc=com"
ldapsearch -x -H ldap://target -b "dc=example,dc=com" "(objectClass=user)"
ldapdomaindump -u '' -p '' target
nmap -p389 --script=ldap-rootdse target
```

RPC

```
rpcclient -U "" target
> enumdomusers / enumdomgroups / querydominfo / netshareenum
python3 lookupsid.py DOMAIN/user:pass@target # impacket
nmap -p135 --script=msrpc-enum target
rpcinfo -p target # Unix/Linux
inix RPC (different protocol)
```